

Analyse MVP lecture de billet avion - React + Vite

Projet Chaye Web Frontend - synthese des recommandations techniques pour le pre-remplissage du formulaire transport.

Objectif

L'objectif est d'aider un utilisateur qui publie une annonce transport a pre-remplir son formulaire a partir d'une photo de billet d'avion. Le frontend lit d'abord un code-barres si disponible, puis utilise l'OCR en fallback si aucun code exploitable n'est detecte.

```
photo billet
  -> reduction image
  -> lecture barcode
  -> fallback OCR
  -> parsing TypeScript
  -> pre-remplissage formulaire transport
  -> confirmation utilisateur
```

Faisabilite Frontend

C'est faisable dans le frontend React + Vite du projet, mais cela doit rester une aide au pre-remplissage, pas une validation automatique. Le resultat doit toujours etre verifie par l'utilisateur avant publication.

Le cas d'usage concerne principalement le formulaire Je t transporte, car il contient deja les champs pertinents : aeroport de depart, aeroport d'arrivee, heure de depart du vol, heure d'arrivee du vol et justificatif de billet.

Formats de codes a lire

QR code

Format tres courant pouvant contenir une URL, du texte ou des donnees structurees. Il est utile a supporter, meme si ce n'est pas toujours le format principal des cartes d'embarquement.

PDF417

Code-barres 2D rectangulaire tres utilise dans les documents de transport, cartes d'embarquement, permis et pieces administratives. Il est pertinent pour les boarding passes.

Aztec

Code 2D compact souvent utilise pour les billets de transport, notamment train, avion et evenements. Il se lit bien meme imprime petit et demande moins de marge blanche qu'un QR code classique.

Data Matrix

Code 2D compact, frequemment utilise en industrie, logistique, sante et documents officiels. Moins visible pour le grand public, mais raisonnable a supporter dans un scanner polyvalent.

Pour un billet d'avion, il faut essayer le barcode avant l'OCR. Si le code est lu, on recupere une donnee structuree, beaucoup plus fiable qu'un texte extrait depuis une photo.

Choix Techniques Recommandes

barcode-detector / ponyfill

Bon choix pour un MVP. Le projet repose sur ZXing-C++ WebAssembly et prend en charge les formats utiles : aztec, data_matrix, pdf417 et qr_code. Derniere release observee : v3.2.0, 1 juin 2026.

tesseract.js

Bon fallback OCR cote navigateur. Le projet est actif, fonctionne en browser et Node, et repose sur WebAssembly. Dernière release observée : v7.0.0, 15 décembre 2025.

zxing-wasm

Alternative solide si l'on veut plus de contrôle sur le chargement du moteur WebAssembly que via le ponyfill barcode-detector. Dernière release observée : v3.1.0, 1 juin 2026.

Compression / redimensionnement image

browser-image-compression est fonctionnel, mais sa dernière release GitHub visible date de mars 2023. Il n'est pas forcément déprécié, mais il ne répond pas idéalement au critère "maintenu régulièrement".

Meilleurs choix pour ce projet : une fonction maison avec canvas / createImageBitmap pour le MVP, ou pica si la qualité et la performance du resize deviennent un sujet.

Stack Proposée Pour Le MVP

- barcode-detector ou zxing-wasm pour lire les codes 2D.
- tesseract.js pour l'OCR fallback.
- Resize image via fonction maison canvas, ou pica si besoin.
- Parsing TypeScript maison, basé sur regex et cas réels.
- Confirmation manuelle obligatoire avant publication.

Limites Du Frontend Seul

- OCR lent sur mobile bas de gamme.
- Photo floue, reflets ou mauvais angle peuvent produire de mauvais résultats.
- Certains billets masquent ou compressent le barcode.
- Confusions OCR possibles : 0 / Ø, I / 1, FDF / FØF.
- Les données lues doivent être confirmées par l'utilisateur.

Pourquoi Un Backend Plus Tard

Pour une version produit très fiable, notamment avec contraintes juridiques, qualité OCR et traçabilité, le traitement devrait passer par le backend AdonisJS.

```
Frontend
  upload temporaire image billet
    ↓
Backend AdonisJS
  appelle Google Cloud Vision / AWS Textract / Azure AI Vision
    ↓
Backend extrait les données
    ↓
Backend supprime l'image brute
    ↓
Frontend reçoit uniquement le JSON pré-rempli
```

La raison principale est la sécurité : une clé Google, AWS ou Azure ne doit jamais être placée dans Vite, car elle serait visible dans le navigateur. Le backend garde la clé secrète, contrôle les accès, peut journaliser les traitements et supprimer l'image brute après analyse.

Conclusion

Pour le projet actuel, le MVP frontend est faisable et pertinent. La bonne stratégie est : barcode d'abord, OCR ensuite, pré-remplissage du formulaire transport, puis validation manuelle. Pour une version juridiquement et produit plus robuste, il faudra déplacer le traitement sensible côté backend.